

SatCLIP: Embedding Extraction Tutorial

Objective

The goal of this experiment is to generate geospatial embeddings using **SatCLIP**, a foundation model that learns geographic representations from latitude and longitude coordinates.

Unlike models such as AlphaEarth, which require multiple Earth observation data sources, SatCLIP only requires geographic coordinates as input:

$$(\text{longitude}, \text{latitude}) \rightarrow \text{SatCLIP} \rightarrow \text{Embedding}$$

Model Information

Model: SatCLIP-ViT16-L40

Repository: Microsoft SatCLIP

Checkpoint:

```
microsoft/SatCLIP-ViT16-L40
```

Embedding dimension:

```
256
```

Environment Setup

A dedicated Conda environment was created to avoid dependency conflicts with other projects.

```
conda create -n satclip python=3.10 -y
conda activate satclip

pip install "numpy==1.26.4"

pip install torch torchvision
pip install lightning
pip install albumentations
pip install huggingface_hub
pip install pandas
pip install rasterio
pip install torchgeo
pip install basemap
pip install scipy==1.11.4

pip install --force-reinstall "numpy==1.26.4"
```

Coordinates Used

The following locations were selected as examples:

Location	Longitude	Latitude
ITAM (Mexico City)	-99.1840	19.3460
Paris	2.3522	48.8566
Tokyo	139.6917	35.6895
New York	-74.0060	40.7128
Monterrey	-100.3161	25.6866

Coordinates must be provided in **longitude-latitude order**.

Embedding Extraction Code

```
from pathlib import Path
import sys

import pandas as pd
import torch
from huggingface_hub import hf_hub_download

sys.path.append(str(Path("satclip").resolve()))

from load import get_satclip

device = "cuda" if torch.cuda.is_available() else "cpu"

locations = [
    ("ITAM_CDMX", -99.1840, 19.3460),
    ("Paris", 2.3522, 48.8566),
    ("Tokyo", 139.6917, 35.6895),
    ("New_York", -74.0060, 40.7128),
    ("Monterrey", -100.3161, 25.6866),
]

coords = torch.tensor(
    [[lon, lat] for _, lon, lat in locations],
    dtype=torch.double,
)

checkpoint = hf_hub_download(
    "microsoft/SatCLIP-ViT16-L40",
    "satclip-vit16-l40.ckpt",
)

model = get_satclip(checkpoint, device=device)
```

```
model.eval()

with torch.no_grad():
    embeddings = model(coords.to(device)).detach().cpu()

print("Embeddings shape:", embeddings.shape)
```

Results

The model successfully generated embeddings for all locations.

```
Embeddings shape: torch.Size([5, 256])
```

This indicates that:

- 5 geographic locations were processed.
- Each location was represented by a 256-dimensional embedding vector.

Therefore, the output matrix has dimensions:

$$5 \times 256$$

Generated Files

The embeddings were exported to CSV format:

```
examples/satclip_example_embeddings.csv
```

Location metadata were also stored:

```
examples/satclip_example_locations.csv
```

These files can be used for:

- Similarity analysis
 - Clustering
 - Dimensionality reduction (PCA, UMAP, t-SNE)
 - Downstream prediction tasks
 - Comparison against other Earth foundation models
-

Interpretation

The individual dimensions of the embedding do not have a direct physical interpretation.

For example, it is not possible to state that:

```
Dimension 17 = vegetation  
Dimension 42 = temperature  
Dimension 89 = population density
```

Instead, the embedding should be viewed as a latent representation learned during contrastive pretraining.

The value of the embedding comes from the relationships between vectors rather than the meaning of any single dimension.

Key Takeaway

This experiment demonstrates that SatCLIP can transform geographic coordinates into dense numerical representations without requiring satellite imagery during inference.

The resulting embeddings provide a compact representation of geographic context that can be used as input for machine learning models and compared against embeddings generated by other Earth foundation models such as AlphaEarth, Clay, Prithvi, GeoCLIP, and MOSAIKS.

Repository used: <https://github.com/microsoft/satclip>